conversion from non-deterministic to deterministic turning machine

Theory of computations

Nada Elkafrawy

What is a Turing Machine?

A Turing Machine is a theoretical computational model used to simulate algorithm execution and solve computational problems

Components:

- Infinite tape
- Read/Write head
- Set of states
- Transition function

Purpose:

- Understand computation theory
- Analyze algorithms and complexity

Deterministic Turing Machine

A Deterministic Turing Machine has exactly one possible action for every state and tape symbol.

$$\delta : Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$$

Characteristics:

- One transition only
- One execution path
- Predictable behavior
- Easier to implement

Example on transition

$\delta(q0,1)=(q1,X,R)$

Meaning:

- Move from q0 to q1
- Write X
- Move Right

Non-Deterministic Turing Machine

A Non-Deterministic Turing Machine can have multiple possible transitions for the same state and input symbol

$$\delta : Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{L, R\})$$

Characteristics:

- Multiple choices
- Multiple execution paths
- The machine explores all possibilities
- Accepts if at least one path accepts

Example on transition

$\delta(q0,1)=\{(q1,X,R)(q2,Y,L)$

If the machine is in q0 and reads 1:

- It may go to q1
- OR go to q2

Both transitions are valid.

DTM vs NDTM

Feature	Number of transitions	Execution paths	Behavior	Practical implementation	Problem solving
DTM	One	Single	Predictable	Realistic	Sequential
NDTM	Multiple	Multiple	Non-deterministic	Theoretical	Parallel-like

Every non-deterministic TM has an equivalent deterministic TM

Proof:

- Given a Nondeterministic TM (N) show how to construct an equivalent Deterministic TM
- If N accepts on any branch, the D will Accept
- If N halts on every branch without any ACCEPT, then D will Halt and Reject.

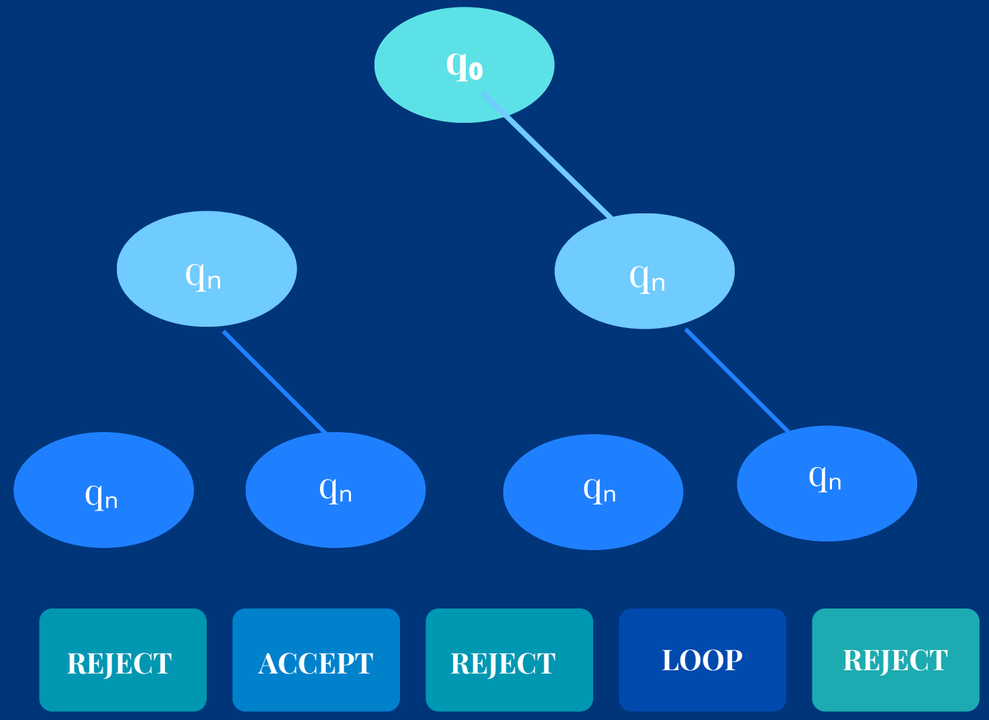
Approach:

- Simulate N
- Simulate all branches of computation
- Search for any way N can Accept

Computational history and Branching Tree

How NTM Computes

1. At each step, the NTM may choose from multiple valid transitions.
2. This creates a tree of computation branches, one per non-deterministic choice.
3. Each branch is an independent computation path with its own tape copy.
4. The NTM ACCEPTS the input if at least one branch reaches the accept state q_acc.
5. The NTM REJECTS only if every branch halts in q_rej or loops forever.
6. Depth of tree = number of steps; branching factor = max choices per step.



↠ NTM ACCEPTS (one branch accepts)

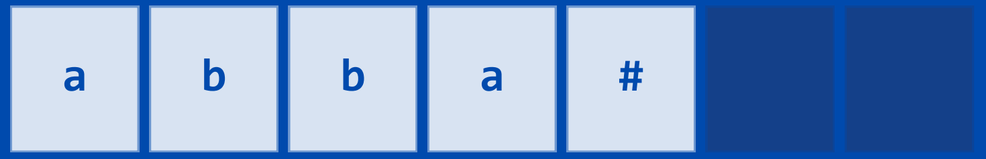
The Three-Tape DTM Simulation

To simulate an NTM, we build a DTM with three tapes that performs a breadth-first search over all NTM branches.

1

INPUT TAPE

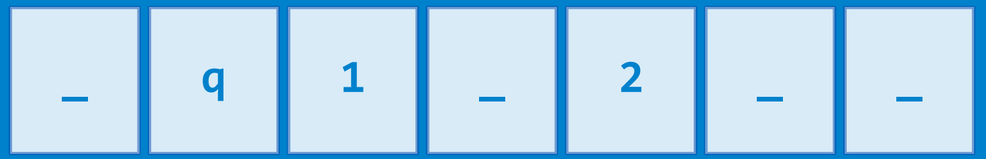
Holds the original input string. Read-only. Never modified. Provides the initial input for each simulated branch.



2

SIMULATION TAPE

Simulates the NTM's tape for the current branch being explored. Contents change per branch. Rewritten for each new branch.



3

ADDRESS TAPE

Tracks which branch is currently being simulated using an address string Each digit selects a non-deterministic choice at each step.

ALGORITHM

steps

1. Copy input string onto Tape 1
2. Initialize Tape 3 with the address string '1' (first branch)
3. Copy Tape 1 onto Tape 2 (fresh simulation tape)
4. Simulate the NTM on Tape 2 using Tape 3 as a guide for non-deterministic choices

steps

5. If simulation reaches q_acc ↠ DTM ACCEPTS and halts
6. If simulation rejects or runs out of choices ↠ go to next address string (BFS order)
7. Generate the next address string on Tape 3 (systematically enumerate all strings over {1..b})
8. Repeat from Step 3. If all finite branches reject ↠ DTM REJECTS

Why BFS and Not DFS

DFS — INCORRECT

If an NTM branch loops infinitely, DFS will follow that branch forever — it never explores other branches that might accept.
Result:
The DTM loops, even if the input should be accepted.

BFS — CORRECT

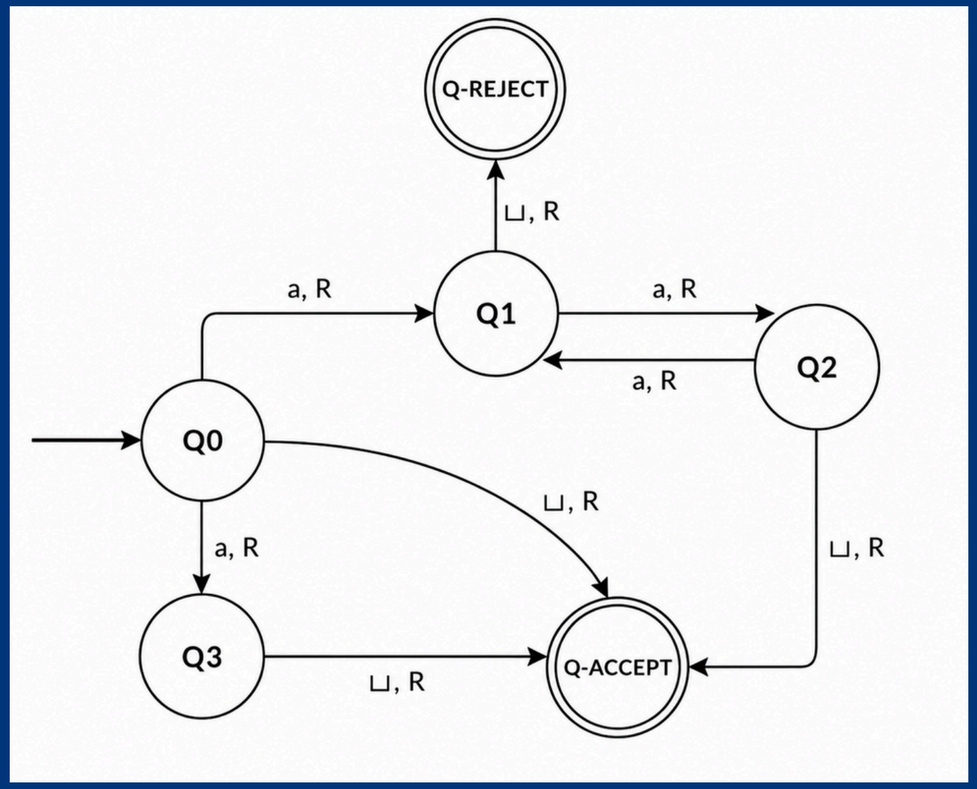
BFS explores all branches level by level (by depth). No single branch can starve others — each gets simulated up to depth d before moving to d+1.
Result:
If any branch accepts, it is eventually found.

Acceptance & Rejection Rules in the Simulation

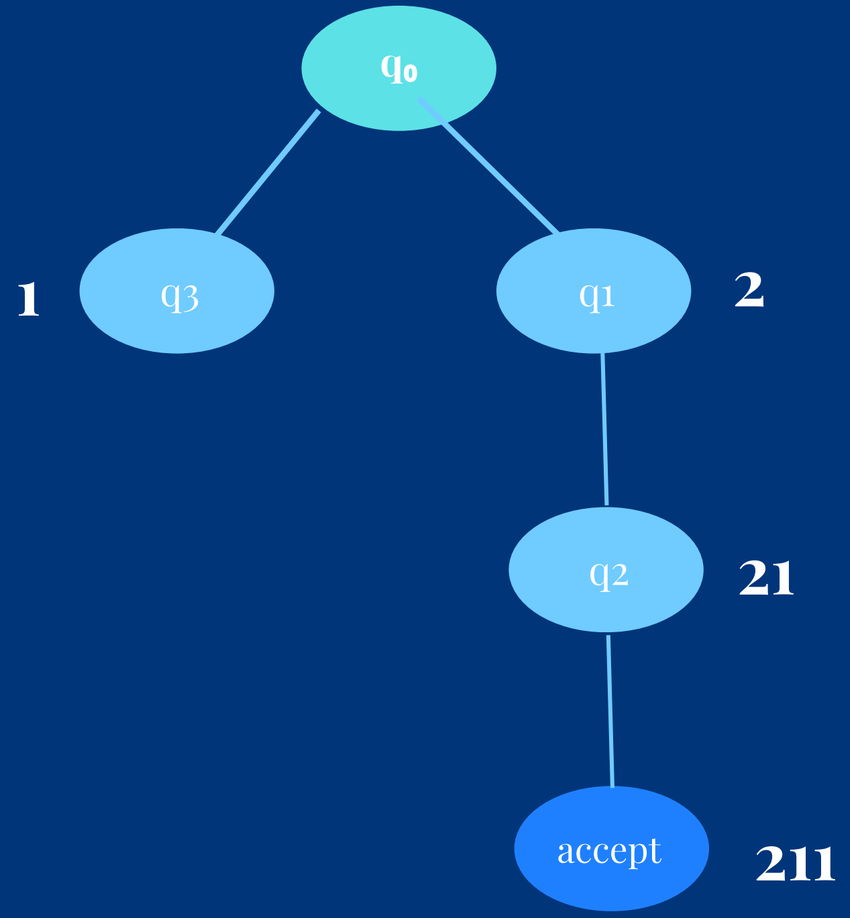
ACCEPT:	The DTM accepts if and only if it finds a branch that reaches q_acc in the NTM.
REJECT:	The DTM rejects if all finite-length branches halt in q_rej (all branches explored and none accept).
LOOP:	The DTM loops if some branches loop infinitely and none accept — it never halts. This is expected behavior for non-deciders.

Example

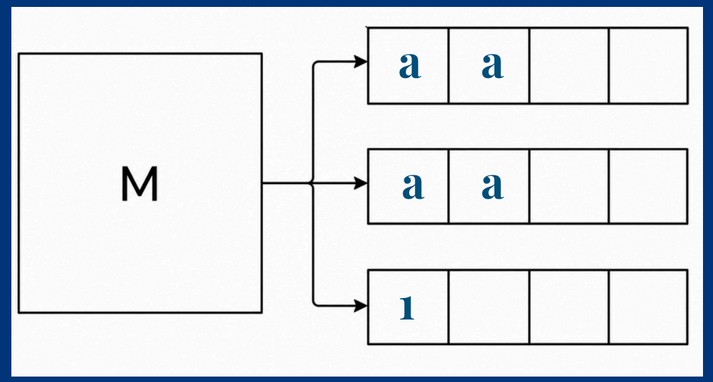
Let L = {ak | k is even, or 1}



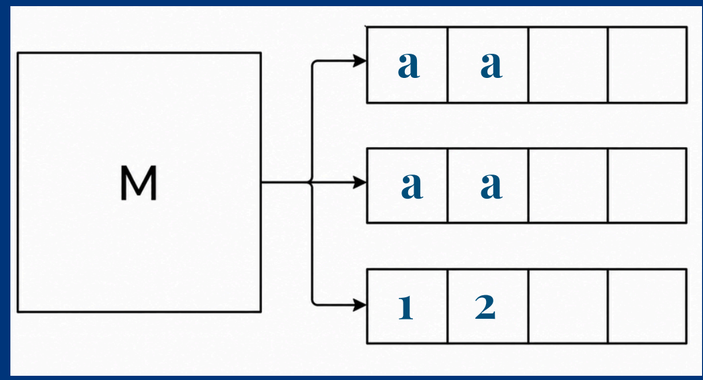
The Non-Deterministic Turing Machine N that decides language L will accept any string of as of length 1, or of an even length



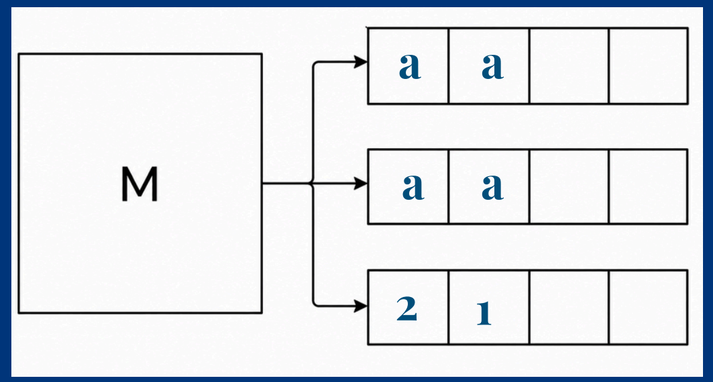
Input : aa



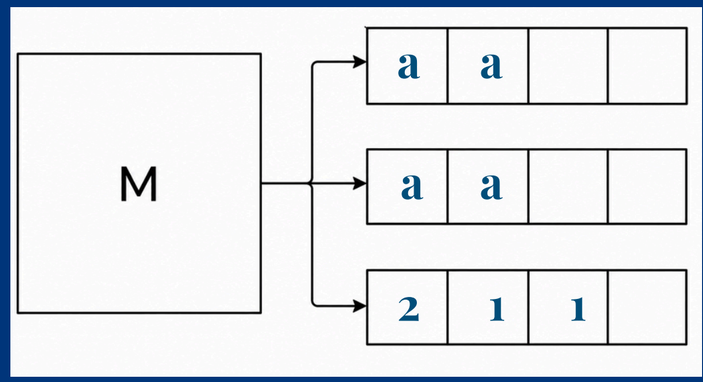
Rejected



Invalid



Run out of symbol



Accepted

Thank You